

Universidad de Mariano Gálvez
Facultad de Ingeniería
Ingeniería en Sistemas de Información
Centro de Antigua Guatemala
Curso: Sistemas Operativos
Catedrático: Ing. JUAN CARLOS OVANDO BORRAYO



Tarea: Simulación procesos en memoria
Fecha de Entrega: 21/08/26

CARNÉ	NOMBRE COMPLETO
1990-23-8990	Gilmar Ricardo Subuyuj Raxon
1990-24-25170	Leonardo Ismael Aspuac Raxon
1990-22-5370	Junior Anibal Tuyuc Cux

1. Descripcion del proyecto

SO-WIKI es un sistema web de arquitectura hibrida desarrollado. El sistema combina dos componentes principales:

- 1) Una base de conocimientos tipo wiki/blog, en la que los usuarios pueden consultar y gestionar articulos tecnicos organizados por categorias, con autenticacion de usuarios y un panel de administracion.
- 2) Un simulador interactivo de gestion de memoria RAM, que representa el comportamiento de un sistema operativo al administrar procesos: creacion de procesos con nombre, memoria requerida y duracion; asignacion de memoria si hay espacio disponible; encolamiento de procesos cuando no hay memoria suficiente; y liberacion automatica de memoria cuando un proceso finaliza su tiempo de ejecucion.

El simulador trabaja con una capacidad total fija de 1024 MB (1 GB) de memoria RAM. Un temporizador (motor de reloj) descuenta un segundo por cada proceso en ejecucion; cuando la duracion restante de un proceso llega a cero, el proceso finaliza y su memoria se libera automaticamente, permitiendo que los procesos en cola de espera puedan pasar a ejecucion si ya existe memoria disponible para ellos.

2. Tecnologias implementadas

2.1 Lenguaje de programacion utilizado

El proyecto esta desarrollado en JavaScript, tanto en el frontend (interfaz de usuario) como en el backend (servidor y API).

2.2 Librerias o frameworks empleados

Frontend (carpeta wiki-frontend):

Libreria / Framework	Uso en el proyecto
React	Construccion de la interfaz de usuario basada en componentes
Vite	Entorno de desarrollo y empaquetado de la aplicacion
React Router DOM	Manejo de rutas y navegacion entre paginas (Home, Simulacion, Articulos, Login, Panel)
React-Bootstrap	Componentes de interfaz basados en Bootstrap
Bootstrap	Estilos y clases de diseno responsivo
Axios	Peticiones HTTP hacia el backend

React Markdown	Renderizado de contenido de articulos escrito en formato Markdown
@supabase/supabase-js	Conexion con Supabase para autentificacion y datos

Backend (carpeta wiki-backend):

Libreria / Framework	Uso en el proyecto
Node.js	Entorno de ejecucion del servidor
Express	Framework para la creacion de la API REST
Nodemon	Reinicio automatico del servidor durante el desarrollo
Cors	Habilitacion de peticiones entre distintos origenes (frontend/backend)
Dotenv	Manejo de variables de entorno (credenciales de Supabase)
@supabase/supabase-js	Conexion del backend con la base de datos Supabase

Base de datos / servicio backend adicional: Supabase (PostgreSQL, autentificacion de usuarios y almacenamiento), utilizado tanto por el frontend como por el backend del proyecto.

3. Instrucciones de instalacion y uso

A continuacion se detallan los pasos para clonar, instalar y ejecutar el proyecto de forma local.

1. Clonar el repositorio del proyecto: `git clone https://github.com/LE0-AR/wiki.git`
2. Configurar el backend: ingresar a la carpeta wiki-backend, crear un archivo .env con las variables SUPABASE_URL y SUPABASE_KEY, instalar las dependencias con `npm install` y levantar el servidor con `npm run dev`.
3. Configurar el frontend: ingresar a la carpeta wiki-frontend, instalar las dependencias con `npm install` y levantar la aplicacion con `npm run dev`.
4. Acceder a la aplicacion desde el navegador en la direccion `http://localhost:5173`
5. Desde el menu de navegacion es posible acceder a Inicio, Blog (articulos) y Simulador(simulador de gestion de memoria RAM).
6. Para usar el simulador: ingresar a la seccion Simulador, completar el formulario Crear Proceso con nombre (opcional), memoria requerida en MB y duracion en segundos, y presionar el boton Agregar a la Cola. El sistema asignara el proceso a ejecucion si existe memoria disponible, o lo colocara en la cola de espera en caso contrario.

Nota: el proyecto tambien se encuentra desplegado en linea en la direccion <https://wikiso.vercel.app/>

4. Capturas de pantalla del programa en funcionamiento

A continuacion se muestra la pagina principal (Home) del sistema, donde se presenta el nombre del proyecto y la descripcion de su arquitectura hibrida.



5. Demostracion visual de procesos en ejecucion y en cola

Para esta demostracion se crearon cinco procesos desde el formulario del simulador: Google Chrome (400 MB, 40 s), VS Code (350 MB, 35 s), Spotify (200 MB, 50 s), Antivirus (300 MB, 20 s) y Backup Service (150 MB, 15 s). Los primeros tres procesos fueron asignados a ejecucion, ya que la memoria RAM disponible alcanzaba para ellos ($400 + 350 + 200 = 950$ MB de 1024 MB totales). Los dos procesos restantes, Antivirus y Backup Service, quedaron en la cola de espera porque ya no habia memoria suficiente para asignarles espacio en ese momento.

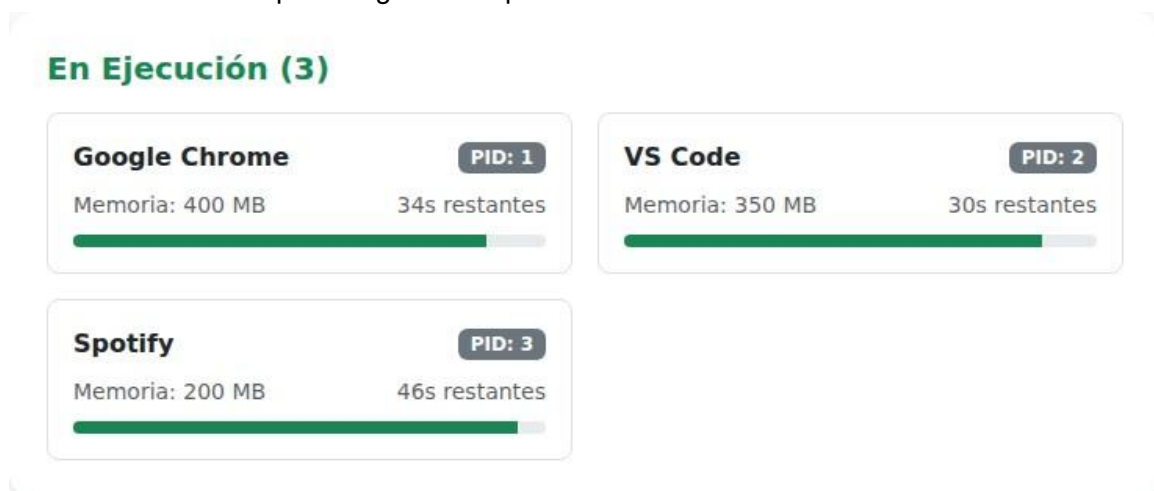


Figura 2. Procesos en ejecucion, cada uno con su PID, memoria asignada y tiempo restante.

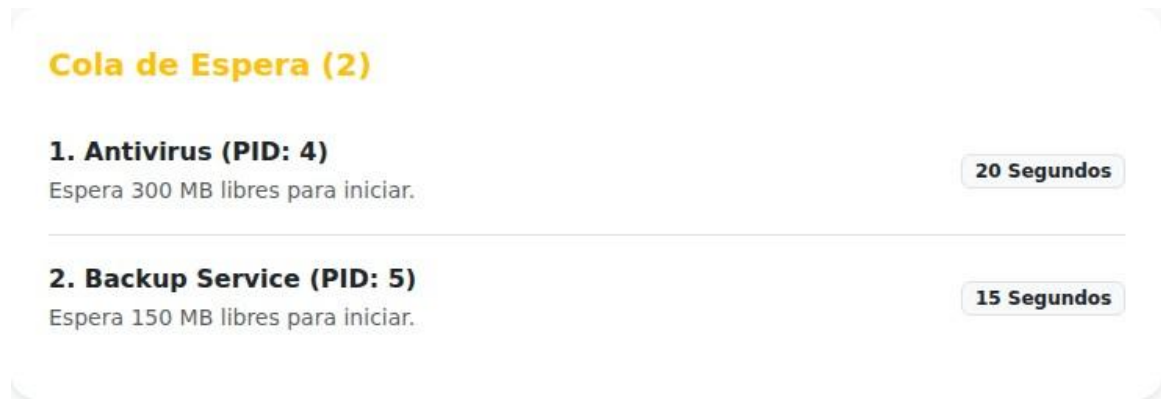


Figura 3. Procesos en cola de espera, indicando la memoria que necesitan liberar para poder iniciar.

6. Estado actual de la memoria RAM

Con los tres procesos en ejecución mencionados anteriormente, el sistema reporta el siguiente estado de memoria: 950 MB usados y 74 MB libres, sobre una capacidad total de 1024 MB, equivalente a un 93% de uso de la memoria RAM.



Figura 4. Estado de la memoria RAM del simulador (950 MB usados, 74 MB libres, 93%).